

Data Engineering Project Definition Guide

Define the project through the Medallion Architecture and the Data Engineering lifecycle. Keep the structure simple, place the greatest attention on the Silver layer, and use the NYC TLC example only as an illustration.

1. Project statement SCOPE

General: define and justify	Example: NYC TLC trip data
• What will the project build? • What problem or analytical need will it address? • What is the intended final outcome? • Who or what will use the result? • What is intentionally outside the project scope?	Build a data pipeline that transforms raw New York City taxi trip records into trusted and consumable datasets for analyzing trip demand and geographic travel patterns. The final outputs support a dashboard or analytical model. Advanced real-time processing is outside the initial scope.

2. Dataset SOURCE AND SUITABILITY

General: define and justify	Example: NYC TLC trip data
• What is the source, format, volume, and refresh pattern? • Why is the dataset suitable for distributed processing? • Which fields and reference datasets are relevant? • What limitations, quality risks, or schema changes are expected? • How will access, licensing, and reproducibility be handled?	Use NYC TLC monthly trip-record files in Parquet format and the taxi-zone lookup data. Trip records describe pickup and drop-off times and locations, distances, fares, payment types, and related attributes. The project should account for source accuracy limitations and possible schema changes.

3. Bronze layer INGEST AND PRESERVE

General: define and justify	Example: NYC TLC trip data
• Which source data must be preserved unchanged? • Which ingestion method and schedule fit the source? • How will files, partitions, and folders be organized? • Which metadata will support traceability and reruns? • How will duplicate ingestion, late data, and schema changes be handled? • How will ingestion be tested and deployed between development and production?	Ingest monthly TLC Parquet files using batch processing. Preserve each source file unchanged and record source name, dataset type, ingestion time, and run identifier. Organize the raw area by dataset and period. Use GitHub Actions to validate changes, deploy the ingestion job, and run it on a regular schedule.

Outcome: Raw data is reproducibly ingested, traceable to its source, and ready for controlled processing.

4. Silver layer PRIMARY FOCUS

General: define and justify	Example: NYC TLC trip data
• Data understanding: What does one record represent? Which fields are essential, optional, derived, or unreliable? • Data model: What is the grain? What keys and relationships are needed? How will history and reference data be represented? • Cleaning: How will missing, duplicate, invalid, inconsistent, or out-of-range values be treated? • Standardization: How will names, types, timestamps, units, and schemas be made consistent? • Enrichment: Which joins, reference data, or derived attributes are needed? • Quality evidence: Which rules and metrics will prove improvement before and after processing? • Implementation: Which transformations should use basic Spark transformations and actions, and where are higher-level APIs appropriate? • Performance: Where are scans, joins, shuffles, skew, or partitioning likely to matter? Which tuning is justified by evidence? • Lifecycle: How will Silver logic be tested, reviewed, versioned, deployed, rerun, and monitored? • Scalability: How would the design respond to larger volumes, late data, or schema evolution?	Use one standardized record per completed trip as the core grain. Validate timestamps, trip duration, distance, fare values, passenger count, and location identifiers. Remove exact duplicates and define whether invalid records are rejected, corrected, or flagged. Enrich trips with pickup and drop-off zone information and derive attributes such as trip duration and calendar fields. Measure missing, duplicate, invalid, accepted, and rejected records before and after processing. Profile the main scans, joins, and aggregations, then apply only evidence-based improvements such as early filtering, suitable partitioning, or reduced shuffling. Test the rules and schemas in GitHub Actions before deployment.

Outcome: A trusted, validated, documented, and reusable dataset that can support multiple downstream uses.

5. Gold layer SERVE THE OUTCOME

General: define and justify	Example: NYC TLC trip data
• Which specific analytical question or consumer need will Gold support? • Why should the Gold structure differ from Silver? • Which dimensions, measures, aggregations, or features are required? • How will correctness and usefulness be demonstrated? • How often should Gold refresh, and how will downstream consumers know it is ready? • What should happen when upstream data is incomplete or fails quality checks?	Create daily or hourly demand measures by pickup zone, with trip count, average duration, average distance, and selected fare measures. Silver retains reusable trip-level records; Gold presents a smaller, purpose-specific structure for direct dashboard use or demand modeling. Validate aggregates against Silver totals and publish only after required quality checks pass.

Outcome: A curated dataset optimized for a defined dashboard, application, report, or analytical model.

6. End-to-end delivery OPERATE THE LIFECYCLE

General: define and justify	Example: NYC TLC trip data
• Can another engineer clone, configure, deploy, and run the project? • Are development and production settings separated? • Do automated checks cover ingestion, transformation, schemas, and quality rules? • Does GitHub Actions support deployment and regular execution? • Are run status, failures, rejected records, and quality results visible? • Can the team explain the main assumptions, risks, trade-offs, and future improvements?	Provide a GitHub repository with notebooks or source code, tests, configuration, workflow files, and run instructions. Separate development and production configuration. Demonstrate an end-to-end scheduled run from Bronze through Gold, including logs and quality results. Document the main design decisions and limits of the solution.

Outcome: A functioning, repeatable data engineering project that another engineer can understand and operate.

Project definition – check-in

Before implementation, prepare a concise 1-2 page definition using these six sections. The Silver section should contain the most detail, followed by Bronze and then Gold.

- A clear project statement and bounded outcome.
- A dataset description covering source, suitability, limitations, and refresh pattern.
- A Bronze design covering ingestion, preservation, traceability, scheduling, and deployment.
- A detailed Silver design covering grain, model, cleaning, quality evidence, Spark implementation, performance, and scalability.
- A focused Gold design tied to a concrete consumer or analytical outcome.
- An end-to-end delivery approach covering reproducibility, environments, automation, monitoring, and ownership.

7. Report requirements EVIDENCE AND REFLECTION

The report should explain the project through its main decisions, implementation evidence, results, and reflection rather than as a chronology of activities.

- Show how source data is understood and converted into a clear, usable structure.
- Demonstrate PySpark programming with high-level DataFrame and Spark SQL APIs.
- Explain and apply efficient Spark coding practices, supported by relevant evidence rather than optimization by assumption.
- Show understanding of an end-to-end data engineering pipeline and Medallion Architecture, adapted to a realistic example.
- Focus the report on the most important design choices, implementation, results, limitations, and lessons learned.
- Reflect meaningfully on your own work from technology perspective: what worked, what did not, why decisions were made, and what should be improved.
- Provide enough code excerpts, models, quality measures, tests, and run evidence to support the claims without turning the report into a code listing.

General UiS grading principle

- **A – Excellent:** An outstanding and clearly distinctive performance, with very strong judgement and a high degree of independence.
- **B – Very good:** A very good performance, with very good judgement and independence.
- **C – Good:** A consistently good performance that is satisfactory in most areas, with good judgement and independence in the most important areas.

DATA ENGINEERING LEARNING PROJECT

- **D – Fair:** An acceptable performance with some significant shortcomings and a certain degree of judgement and independence.
- **E – Sufficient:** Meets the minimum academic requirements, but shows limited judgement and independence.
- **F – Fail:** Does not meet the minimum academic requirements and shows insufficient judgement and independence.